



Manual del motor de navegación de NPCs

Del archivo de texto al camino óptimo: cómo funciona cada archivo del repositorio, cómo se conectan y cómo piensan BFS, Dijkstra, el voraz y A^ .*

EQUIPO

Victor Paredes Maza
Diana Carolina Li Gayoso
Jahat Jassiel Trinidad León

REPOSITORIO

engine/ · web/ · unity/ · docs/
 Package Python `npc_nav` 0.2.0

VERSIÓN

29 de septiembre de 2026
Documento para uso interno del
equipo

Índice

Parte I · La idea completa

1. Qué resuelve el motor	3
2. Las seis estaciones	5
3. Cómo se conectan los archivos	7

Parte II · Del texto al grafo

4. Los datos: el mapa y el solucionario	11
5. La lectura: infrastructure/	13
6. El dominio: domain/	16

Parte III · Los algoritmos

7. El contrato, el catálogo y el observador	21
8. El esqueleto común y el mapa de ejemplo	23
9. BFS — búsqueda en anchura	24
10. Dijkstra	27
11. Voraz primero el mejor	30
12. A*	33
13. UFDS — regiones conectadas	36
14. Los cuatro, frente a frente	38

Parte IV · Ejecutar, medir y mostrar

15. La ejecución y las métricas	41
16. Recorrido de npc-nav comparar	43
17. Cómo se dibuja la imagen	45
18. La verificación contra el solucionario	46

Parte V · Referencia

19. Comandos y puesta en marcha	49
20. Pruebas automatizadas	50
21. Glosario	51
22. Referencias	52

I

La idea completa

Qué resuelve el motor, por qué estaciones pasa la información y cómo se conectan los archivos entre sí.

1 Qué resuelve el motor

2 Las seis estaciones

3 Cómo se conectan los archivos

Qué resuelve el motor

Un NPC tiene que ir de un punto a otro de un mapa real de Dragon Age. El motor convierte ese mapa en un grafo, busca el camino con cuatro algoritmos distintos y mide cuánto trabajó cada uno.

Todo el proyecto gira alrededor de una pregunta: *¿cuál es el camino más corto entre dos celdas, y cuánto cuesta encontrarlo?* La primera mitad se responde con un algoritmo correcto. La segunda es la que interesa al curso de Complejidad Algorítmica: dos algoritmos pueden dar el mismo camino con una diferencia de trabajo de tres veces o más.

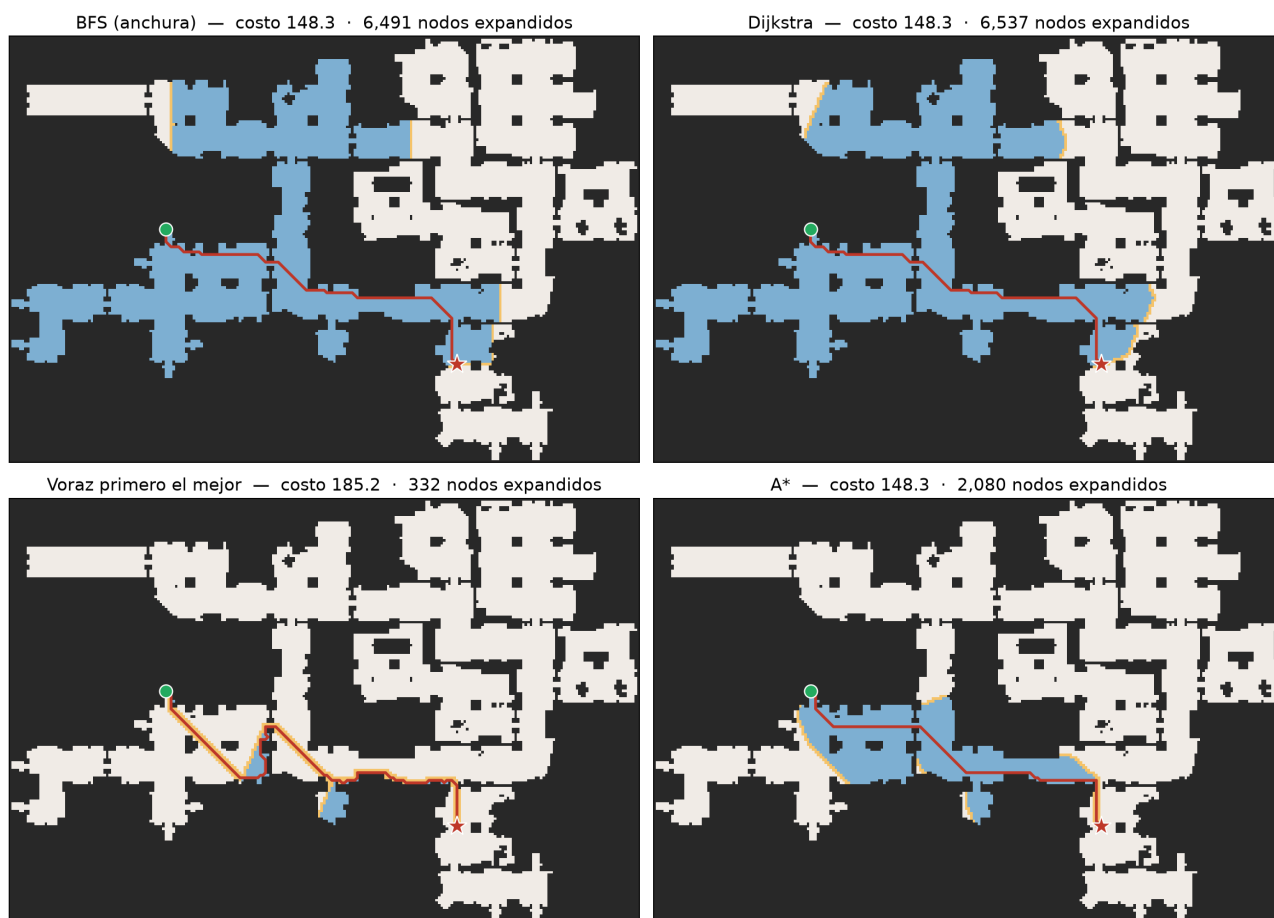


Figura 1.1 · Mapa den011d, caso 375. Cada panel es un algoritmo resolviendo la misma misión. Azul: celdas que el algoritmo examinó. Amarillo: celdas que vio pero dejó pendientes. Rojo: el camino final. Verde (●): inicio. Estrella (★): destino.

ÓPTIMO PUBLICADO

148.25

A* · EXPANDIDAS

2 080

DIJKSTRA ·
EXPANDIDAS

6 537

VORAZ · COSTO

185.17

La figura resume el trabajo. **BFS** y **Dijkstra** encuentran el camino óptimo pero exploran a ciegas medio mapa. El **voraz** casi no explora, pero su camino es un 25 % más largo porque se deja llevar por la dirección

del destino y choca con los muros. **A*** encuentra el mismo camino óptimo examinando un tercio de lo que necesita Dijkstra.

IDEA CLAVE

Para un NPC que debe recalcular su ruta cada vez que aparece un obstáculo, las celdas examinadas son el costo de pensar. Reducirlas sin perder la ruta óptima es exactamente lo que el caso pide con «eficiente en tiempo real».

Cómo leer este manual

La **Parte I** da el mapa general: por dónde pasa la información y qué archivo conoce a cuál. La **Parte II** muestra cómo un archivo de texto se convierte en un grafo. La **Parte III** explica cada algoritmo con su pseudocódigo, su código real y un ejemplo pequeño resuelto celda por celda. La **Parte IV** sigue un comando de principio a fin hasta producir la tabla y la imagen. La **Parte V** es referencia: comandos, pruebas y glosario.

Todo el código que aparece aquí es el código real del repositorio, con los números de línea de cada archivo.

Las seis estaciones

El motor funciona como una línea de ensamblaje: entra un archivo de texto y sale una tabla o una imagen. Cada estación vive en su propia carpeta y hace una sola cosa.



Figura 2.1 · Flujo del motor. Entre cada estación se indica qué dato viaja. La estación resaltada es la que el equipo expone en la sustentación.

ESTACIÓN	CARPETA	RECIBE	ENTREGA
1 · Datos	<code>engine/data/</code>	—	Archivos <code>.map</code> y <code>.scen</code>
2 · Lectura	<code>infrastructure/</code>	Rutas de archivo	<code>Mapa</code> y <code>Escenario</code>
3 · Grafo	<code>domain/</code>	<code>Mapa</code>	<code>SesionNavegacion</code> con <code>vecinos()</code>
4 · Algoritmos	<code>algorithms/</code>	Grafo, inicio, destino, observador	<code>ResultadoBusqueda</code> + avisos al observador
5 · Ejecución	<code>application/</code>	Sesión y nombre del algoritmo	<code>Ejecucion</code> con métricas y traza
6 · Salida	<code>cli/</code> , <code>tools/</code> , <code>api/</code>	<code>Ejecucion</code>	Tabla, imagen o JSON

Dónde está cada cosa

```
complejidad-npc-ai/
├─ engine/
│   ├── data/
│   └── src/npc_nav/
│       ├── infrastructure/
│       ├── domain/
│       ├── algorithms/
│       ├── application/
│       ├── cli/
│       └── api/
├─ tools/
├─ tests/
└─ results/

web/
unity/
docs/
```

motor en Python

1 · mapas (.map) y solucionario (.scen)

2 · movingai.py · repositorio.py · exportadores.py

3 · mapa.py · escenario.py · movimiento.py · sesion.py

4 · contrato.py · catalogo.py · bfs.py · dijkstra.py
voraz.py · astar.py · ufds.py

5 · busqueda.py · analisis.py · validacion.py · verificacion.py

6 · main.py (comando npc-nav)

6 · FastAPI (siguiente rama)

6 · visualizar_busqueda.py · tb1/visualizar_grafo.py

unit/ · integration/

evidencia: validación y verificación

dashboard Vue (siguiente rama)

cliente 3D (hito 2)

arquitectura · plan · adr · figuras · informe

Cómo se conectan los archivos

Cada archivo solo importa archivos de capas más internas. Nada dentro de `domain/` sabe que existen los algoritmos, y ningún algoritmo sabe que existen los archivos `.map`, la línea de comandos o la web.

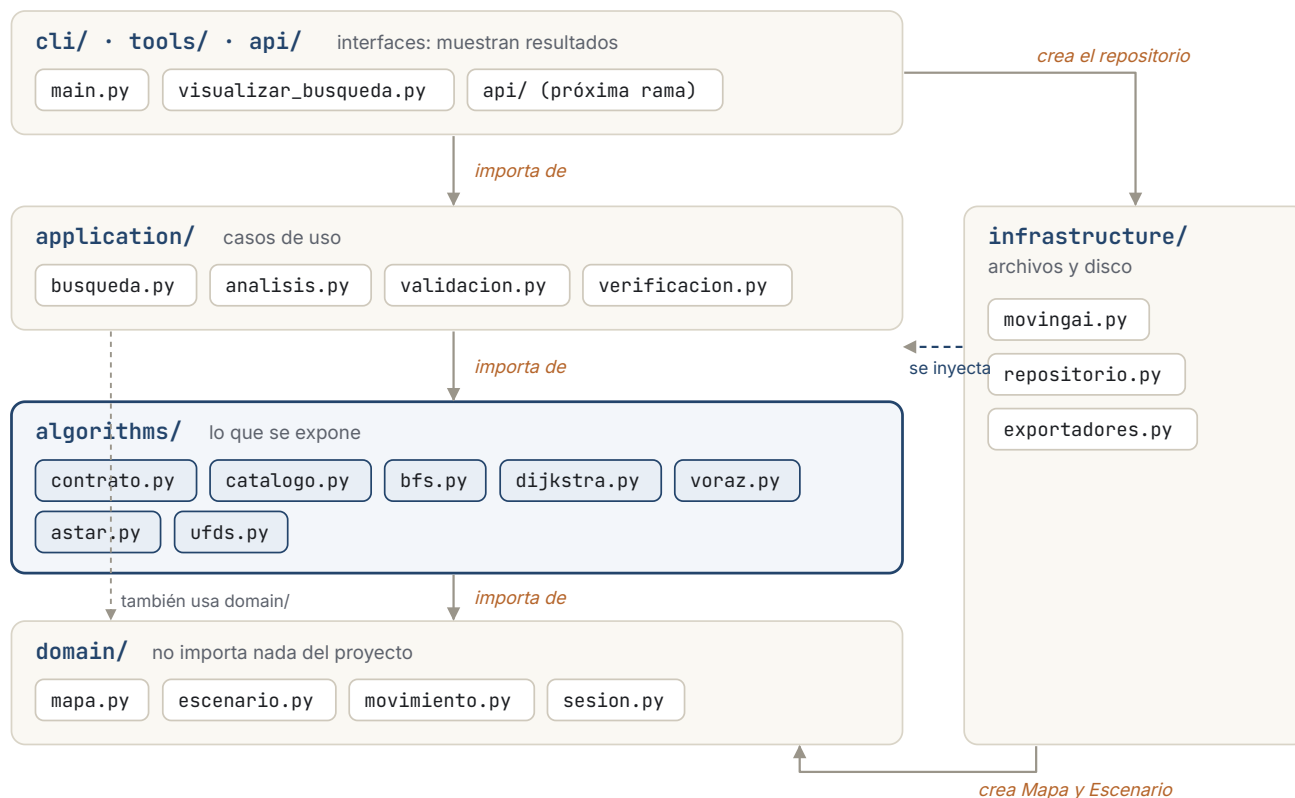


Figura 3.1 · Capas y dependencias reales. Las flechas indican «importa de». Se extrajeron automáticamente del código con el módulo `ast` de Python, así que reflejan el repositorio tal como está.

POR QUÉ IMPORTA ESTA REGLA

Como los algoritmos solo dependen de `domain/`, se pueden probar con un mapa de 3×3 escrito a mano dentro de la prueba, sin archivos. Y si mañana los mapas vienen de una base de datos o de Unity, solo cambia `infrastructure/`: los algoritmos no se tocan.

Qué importa cada archivo

ARCHIVO	IMPORTA (MÓDULO → NOMBRES)
<code>domain/escenario.py</code>	<code>domain.mapa</code> → Posicion
<code>domain/mapa.py</code>	nada del proyecto
<code>domain/movimiento.py</code>	<code>domain.mapa</code> → Posicion
<code>domain/sesion.py</code>	<code>domain.mapa</code> → Mapa, Posicion <code>domain.movimiento</code> → movimientos_desde

ARCHIVO	IMPORTA (MÓDULO → NOMBRES)
infrastructure/exportadores.py	nada del proyecto
infrastructure/movingai.py	domain.escenario → Escenario domain.mapa → Mapa
infrastructure/repositorio.py	domain.escenario → Escenario domain.mapa → Mapa infrastructure.exportadores → sha256 infrastructure.movingai → leer_escenarios, leer_mapa
algorithms/astar.py	domain.movimiento → distancia_octil algorithms.contrato → NULO, SIN_RUTA, ResultadoBusqueda, reconstruir_ruta
algorithms/bfs.py	algorithms.contrato → NULO, SIN_RUTA, ResultadoBusqueda, reconstruir_ruta
algorithms/catalogo.py	algorithms.astar → astar algorithms.bfs → bfs algorithms.contrato → Algoritmo algorithms.dijkstra → dijkstra algorithms.voraz → voraz
algorithms/contrato.py	domain.mapa → Posicion
algorithms/dijkstra.py	algorithms.contrato → NULO, SIN_RUTA, ResultadoBusqueda, reconstruir_ruta
algorithms/ufds.py	nada del proyecto
algorithms/voraz.py	domain.movimiento → distancia_octil algorithms.contrato → NULO, SIN_RUTA, ResultadoBusqueda, reconstruir_ruta
application/analisis.py	algorithms.ufds → regiones_conectadas domain.mapa → Mapa, Posicion domain.sesion → SesionNavegacion
application/busqueda.py	algorithms.catalogo → obtener domain.mapa → Posicion domain.sesion → SesionNavegacion
application/validacion.py	algorithms.ufds → regiones_conectadas domain.movimiento → distancia_octil domain.sesion → SesionNavegacion application.analisis → construir_grafo
application/verificacion.py	algorithms.catalogo → obtener domain.sesion → SesionNavegacion application.busqueda → ejecutar_busqueda application.validacion → validar_dataset

ARCHIVO**IMPORTA (MÓDULO → NOMBRES)**

`cli/main.py``npc_nav → algorithms``algorithms.catalogo → ALGORITMOS, obtener``application → comparar_algoritmos, ejecutar_búsqueda, estadísticas_grafo,
validar_dataset, verificar_benchmark``domain.sesion → SesionNavegacion``infrastructure.exportadores → guardar_csv, guardar_json, sha256``infrastructure.repositorio → RepositorioDataset`

Los archivos `__init__.py` solo reexportan nombres para acortar los imports; por ejemplo, `from npc_nav.algorithms import astar` en lugar de `from npc_nav.algorithms.astar import astar`.

II

Del texto al grafo

Cómo un archivo con arrobas y puntos termina siendo un grafo ponderado que cualquier algoritmo puede recorrer.

4 Los datos: el mapa y el solucionario

5 La lectura: infrastructure/

6 El dominio: domain/

Los datos: el mapa y el solucionario

Cada mapa son dos archivos de texto de Moving AI Lab: la grilla del nivel y una lista de misiones con su respuesta correcta.

El mapa: `maps/den011d.map`

Cuatro líneas de cabecera y después una fila de texto por cada fila del nivel. Cada carácter es una celda.

```
type octile      ← conectividad de 8 direcciones
height 167      ← filas
width 247       ← columnas
map
@@@@@TT..TT...TT...TTTT...TTTTTT...TTTT.....
@@@@@T.....TTT...TTTTTT.....
@@@@@.....TT.....
```

SÍMBOLO	SIGNIFICADO	¿SE PUEDE CAMINAR?
. G	Suelo	Sí: es un vértice del grafo
@ 0	Fuera del mapa	No
T	Árbol	No
S W	Pantano y agua (no aparecen en nuestros mapas)	No, en este proyecto

MAPA	TAMAÑO	VÉRTICES	ARISTAS	REGIONES	MISIONES
den011d	167 × 247	14 506	52 710	1	750
brc201d	388 × 391	25 645	92 042	167	2 090
brc100d	492 × 512	31 023	115 524	1	1 360

El solucionario: `scenarios/den011d.map.scen`

Cada línea es una misión. Moving AI calculó el camino más corto de cada una con su propio software y publicó el costo, así que para 4 200 misiones sabemos de antemano cuál es la respuesta correcta. Así se lee la misión del **caso 375**:

```
37 den011d.map 247 167 61 75 175 128 148.25483398
```

37	den011d.map	247 167	61 75	175 128	148.25483398
bucket: grupo de misiones de longitud parecida	mapa al que pertenece	ancho y alto, para comprobar que coincide	inicio en (x, y) = columna 61, fila 75	destino: columna 175, fila 128	costo óptimo: la respuesta correcta

CUIDADO CON LAS COORDENADAS

El archivo usa (x, y) , que es (columna, fila). Todo el código usa $(fila, columna)$. La conversión se hace una sola vez, al leer: el inicio $(x=61, y=75)$ se guarda como $(75, 61)$. Por eso el primer evento de la traza del caso 375 es la posición $[75, 61]$.

Los casos se cuentan desde 0 en el orden del archivo. La primera línea es la cabecera `version 1`, así que el caso 375 está en la línea 377.

La lectura

Tres archivos se encargan de todo lo que toca el disco: leer y validar el formato de Moving AI, encontrar los archivos por nombre y escribir la evidencia.

movingai.py — leer y validar

`leer_mapa` revisa cada parte de la cabecera y rechaza cualquier archivo mal formado con un mensaje que dice qué falló. Si todo está bien, devuelve un `Mapa` con las filas de terreno.

src/npc_nav/infrastructure/movingai.py

Python · código real del repositorio

```

14 def leer_mapa(ruta: str | Path) → Mapa:
15     ruta = Path(ruta)
16     lineas = ruta.read_text(encoding='utf-8-sig').splitlines()
17     try:
18         if len(lineas) < 4 or lineas[0].split() ≠ ['type', 'octile']:
19             raise ValueError('Se requiere una cabecera type octile')
20         dimensiones = []
21         for linea, clave in zip(lineas[1:3], ('height', 'width')):
22             partes = linea.split()
23             if len(partes) ≠ 2 or partes[0] ≠ clave:
24                 raise ValueError(f'Cabecera {clave} inválida')
25             dimensiones.append(int(partes[1]))
26         alto, ancho = dimensiones
27         if alto ≤ 0 or ancho ≤ 0 or lineas[3].strip() ≠ 'map':
28             raise ValueError('Dimensiones o marcador map inválidos')
29         if len(lineas[4:]) ≠ alto:
30             raise ValueError(f'Se esperaban {alto} filas de terreno')
31         if any(len(fila) ≠ ancho for fila in lineas[4:]):
32             raise ValueError(f'Se esperaban {ancho} columnas por fila')
33         return Mapa(ruta.name, tuple(lineas[4:]))
34     except ValueError as error:
35         raise ValueError(f'{ruta}: {error}') from error

```

- ❶ `utf-8-sig` tolera archivos guardados con marca BOM en Windows; `splitlines()` separa las filas sin importar si terminan en `\n` o `\r\n`.
- ❷ Cada comprobación lanza `ValueError` con una frase concreta. El `except` final le antepone la ruta del archivo, así el error dice exactamente qué archivo falló.
- ❸ `lineas[4:]` son las filas de terreno: todo después de la cabecera de cuatro líneas.

`leer_escenarios` hace lo mismo con el `.scen`. Aquí está la conversión de coordenadas: observa que `inicio` toma `partes[5]` (y) antes que `partes[4]` (x).

src/npc_nav/infrastructure/movingai.py

Python · código real del repositorio

```

38 def leer_escenarios(ruta: str | Path) → tuple[Escenario, ...]:
39     ruta = Path(ruta)
40     lineas = ruta.read_text(encoding='utf-8-sig').splitlines()
41     if not lineas or lineas[0].split() not in (['version', '1'], ['version', '1.0']):
42         raise ValueError(f'{ruta}: se requiere version 1 o version 1.0')

```

```

43     casos = []
44     for numero, linea in enumerate(lineas[1:], start=2):
45         partes = linea.split()
46         if not partes:
47             continue
48         try:
49             if len(partes) != 9:
50                 raise ValueError('Se requieren nueve campos')
51             casos.append(Escenario(
52                 bucket=int(partes[0]), mapa=partes[1],
53                 ancho=int(partes[2]), alto=int(partes[3]),
54                 inicio=(int(partes[5]), int(partes[4])),
55                 destino=(int(partes[7]), int(partes[6])),
56                 optimo=float(partes[8]),
57             ))
58         except ValueError as error:
59             raise ValueError(f'{ruta}, línea {numero}: {error}') from error
60     if not casos:
61         raise ValueError(f'{ruta}: no contiene escenarios')
62     return tuple(casos)

```

repositorio.py — el bibliotecario

El resto del proyecto nunca escribe rutas de archivo: le pide un mapa por su nombre al repositorio. Él arma la ruta, lee el archivo una sola vez y lo guarda en caché con `tru_cache`.

src/npc_nav/infrastructure/repositorio.py

Python · código real del repositorio

```

24 def carpeta_datos() → Path:
25     return Path(os.environ.get('NPC_DATA_DIR', CARPETA_POR_DEFECTO))
26
27
28 class RepositorioDataset:
29     def __init__(self, carpeta: str | Path | None = None):
30         self.carpeta = Path(carpeta) if carpeta else carpeta_datos()
31
32     def ruta_mapa(self, nombre: str) → Path:
33         return self.carpeta / 'maps' / f'{self._normalizar(nombre)}.map'
34
35     def ruta_escenarios(self, nombre: str) → Path:
36         return self.carpeta / 'scenarios' / f'{self._normalizar(nombre)}.map.scen'
37
38     def listar_mapas(self) → list[str]:
39         """Nombres sin extensión, p. ej. ['brc100d', 'brc201d', 'den011d']. """
40         return sorted(ruta.stem for ruta in (self.carpeta / 'maps').glob('*map'))
41
42     def listar_escenarios(self) → list[str]:
43         """Nombres de mapa que tienen archivo .map.scen. """
44         return sorted(ruta.name[:-len('.map.scen')]
45                       for ruta in (self.carpeta / 'scenarios').glob('*map.scen'))
46
47     def huellas(self, nombre: str) → dict[str, str]:
48         """SHA-256 del mapa y de sus escenarios, para identificar los datos usados. """
49         return {'sha256_mapa': sha256(self.ruta_mapa(nombre)),
50               'sha256_escenarios': sha256(self.ruta_escenarios(nombre))}
51
52     def mapa(self, nombre: str) → Mapa:
53         return _leer_mapa_cacheado(self.ruta_mapa(nombre))
54

```

```

55     def escenarios(self, nombre: str) → tuple[Escenario, ...]:
56         return _leer_escenarios_cacheado(self.ruta_escenarios(nombre))
57
58     def escenario(self, nombre: str, indice: int) → Escenario:
59         casos = self.escenarios(nombre)
60         if not 0 ≤ indice < len(casos):
61             raise ValueError(f'Índice de caso fuera del rango 0..{len(casos) - 1}')
62         return casos[indice]
63
64     @staticmethod
65     def _normalizar(nombre: str) → str:
66         nombre = Path(nombre).name
67         for sufijo in ('.scen', '.map'):
68             if nombre.endswith(sufijo):
69                 nombre = nombre[:-len(sufijo)]
70         return nombre

```

- 1 `CARPETA_POR_DEFECTO` sube tres carpetas desde este archivo hasta `engine/` y entra a `data/`. Funciona sin importar desde dónde se ejecute el comando.
- 2 `NPC_DATA_DIR` permite usar otra carpeta de datos sin tocar código; las pruebas usan carpetas temporales pasándolas al constructor.
- 3 `_normalizar` acepta `'den011d'`, `'den011d.map'` o una ruta completa, y siempre busca el mismo archivo.
- 4 `huellas` calcula el SHA-256 de los archivos: queda registrado en la evidencia qué datos exactos se usaron.

`exportadores.py` tiene tres funciones pequeñas: `guardar_json`, `guardar_csv` y `sha256`. Son las únicas que escriben en disco.

El dominio: del mapa al grafo

Nadie construye el grafo completo por adelantado. El grafo existe a través de una sola pregunta: *¿quiénes son los vecinos de esta celda y cuánto cuesta ir a cada uno?*

CONCEPTO DEL CURSO	EN EL CÓDIGO
Vértice	Una celda caminable, identificada por la tupla <code>(fila, columna)</code>
Arista	Un movimiento legal a una celda vecina
Peso	<code>1.0</code> en recto, <code>sqrt(2)</code> $\approx$ 1.414 en diagonal
Lista de adyacencia	No se guarda: <code>SesionNavegacion.vecinos(celda)</code> la calcula al momento

A esto se le llama **grafo implícito**. Tiene dos ventajas: no gasta memoria en 260 000 aristas que la mayoría de las búsquedas nunca mira, y cuando aparece un obstáculo no hay ninguna lista vieja que actualizar.

mapa.py — el terreno

src/npc_nav/domain/mapa.py

Python · código real del repositorio

```

19 @dataclass(frozen=True)
20 class Mapa:
21     nombre: str
22     celdas: tuple[str, ...]
23
24     def __post_init__(self):
25         object.__setattr__(self, 'celdas', tuple(self.celdas))
26         if not self.celdas or not self.celdas[0]:
27             raise ValueError('El mapa debe tener dimensiones positivas')
28         if any(len(fila) != self.ancho for fila in self.celdas):
29             raise ValueError('Las filas del mapa deben tener el mismo ancho')
30         if any(celda not in TERRENOS for fila in self.celdas for celda in fila):
31             raise ValueError('El mapa contiene un símbolo de terreno desconocido')
32
33     @property
34     def alto(self) → int:
35         return len(self.celdas)
36
37     @property
38     def ancho(self) → int:
39         return len(self.celdas[0])
40
41     def contiene(self, posicion: Posicion) → bool:
42         fila, columna = posicion
43         return 0 ≤ fila < self.alto and 0 ≤ columna < self.ancho
44
45     def transitable(self, posicion: Posicion) → bool:
46         return (self.contiene(posicion)
47                 and self.celdas[posicion[0]][posicion[1]] in TRANSITABLES)
48
49     def posiciones_transitables(self):

```

```

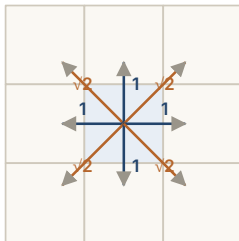
50     """Recorre todas las celdas transitables (los vértices del grafo)."""
51     for fila, texto in enumerate(self.celdas):
52         for columna, celda in enumerate(texto):
53             if celda in TRANSITABLES:
54                 yield fila, columna

```

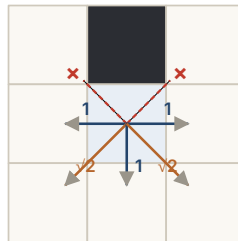
- 1 `@dataclass(frozen=True)` hace al mapa **inmutable**: nadie puede modificar el terreno por accidente. Los obstáculos dinámicos viven en otro lugar (la sesión).
- 2 `__post_init__` valida al construir: filas del mismo ancho y solo símbolos conocidos.
- 3 `transitable` es la pregunta más usada del proyecto: dentro de la grilla y con `.` o `G`.

movimiento.py — las reglas del juego

Sin obstáculos: 8 aristas



Pared al norte: 5 aristas



La pared bloquea el paso recto al norte y también las dos diagonales del norte: cruzarlas sería atravesar la esquina de la pared.

— recto · costo 1 — diagonal · $\sqrt{2} \approx 1.414$

Figura 6.1 · Las ocho aristas de una celda. Izquierda: sin obstáculos hay 4 movimientos rectos de costo 1 y 4 diagonales de costo $\sqrt{2}$. Derecha: con una pared al norte, las dos diagonales que pasarían por su esquina dejan de ser legales.

src/npc_nav/domain/movimiento.py

Python · código real del repositorio

```

21 def movimientos_desde(origen: Posicion, transitable: Callable[[Posicion], bool]):
22     """Devuelve pares (vecino, costo); una celda bloqueada no tiene movimientos."""
23     if not transitable(origen):
24         return ()
25     fila, columna = origen
26     movimientos = []
27     for df, dc in CARDINALES:
28         destino = (fila + df, columna + dc)
29         if transitable(destino):
30             movimientos.append((destino, COSTO_CARDINAL))
31     for df, dc in DIAGONALES:
32         destino = (fila + df, columna + dc)
33         if (transitable(destino) and transitable((fila + df, columna))
34             and transitable((fila, columna + dc))):
35             movimientos.append((destino, COSTO_DIAGONAL))
36     return tuple(movimientos)
37
38
39 def distancia_octil(a: Posicion, b: Posicion) -> float:
40     """Costo del trayecto más corto sin obstáculos: heurística admisible y consistente.
41
42     h = max(df, dc) + (raíz de 2 - 1) * min(df, dc)
43     """
44     df, dc = abs(a[0] - b[0]), abs(a[1] - b[1])
45     return max(df, dc) + (COSTO_DIAGONAL - 1) * min(df, dc)

```

- 1 Si la celda de origen está bloqueada, no tiene aristas.

- 2 Primer bucle: las cuatro direcciones rectas con costo 1.
- 3 Segundo bucle: una diagonal exige que estén libres el destino y las dos celdas laterales. Es la regla con la que Moving AI calculó los óptimos: sin ella nuestros costos no coincidirían con el solucionario.
- 4 `transitable` llega como parámetro (una función). Así la misma regla sirve para el mapa original y para una sesión con obstáculos.

LA HEURÍSTICA OCTIL

`distancia_octil` es la estimación de «cuánto me falta» que usan A* y el voraz. Es la distancia exacta si no hubiera ningún obstáculo: avanzar en diagonal todo lo posible y completar en recto.

$$h(a, b) = \max(\Delta\text{fila}, \Delta\text{columna}) + (\sqrt{2} - 1) \cdot \min(\Delta\text{fila}, \Delta\text{columna})$$

Ejemplo: de (75, 61) a (128, 175) → $\Delta\text{fila} = 53$, $\Delta\text{columna} = 114$
 $h = 114 + 0.4142 \cdot 53 = 135.95$ (el costo real es 148.25)

Dos propiedades la hacen segura para A*. Es **admisibile**: nunca sobreestima, porque los obstáculos solo pueden alargar el camino. Es **consistente**: al dar un paso de costo w , la estimación nunca baja más que w .

sesion.py — el mapa durante una partida

La `SesionNavegacion` combina el mapa inmutable con los obstáculos que aparecen durante el juego. Es el grafo que reciben todos los algoritmos.

src/npc_nav/domain/sesion.py

Python · código real del repositorio

```

32     def transitable(self, posicion: Posicion) → bool:
33         return self.mapa.transitable(posicion) and posicion not in self._bloqueadas
34
35     def vecinos(self, origen: Posicion) → tuple[tuple[Posicion, float], ...]:
36         """Aristas salientes de `origen`: pares (vecino, costo)."""
37         return movimientos_desde(origen, self.transitable)
38
39
40     def costo_ruta(self, ruta: Iterable[Posicion]) → float:
41         """Suma los pesos de una ruta y rechaza saltos o pasos bloqueados."""
42         puntos = tuple(ruta)
43         if not puntos:
44             raise ValueError('Una ruta debe contener al menos una posición')
45         self.validar_consulta(puntos[0], puntos[-1])
46         total = 0.0
47         for origen, destino in zip(puntos, puntos[1:]):
48             costo = self.costo_movimiento(origen, destino)
49             if costo == inf:
50                 raise ValueError(f'Paso de ruta inválido: {origen} → {destino}')
51             total += costo
52         return total
53
54
55     def actualizar_obstaculos(self, *, bloquear: Iterable[Posicion] = (),
56                               liberar: Iterable[Posicion] = ()) → tuple[Posicion, ...]:
57         """Aplica un cambio atómico y devuelve las celdas que realmente cambiaron.
58
59         Solo se puede bloquear o liberar terreno originalmente transitable.
60         """
61         bloquear, liberar = set(bloquear), set(liberar)
62         if bloquear & liberar:

```

```

72         raise ValueError('Una celda no puede bloquearse y liberarse en el mismo cambio')
73     for posicion in bloquear | liberar:
74         if not self.mapa.transitable(posicion):
75             raise ValueError(f'Obstáculo fuera del terreno transitable: {posicion}')
76     nuevas = (self._bloqueadas | bloquear) - liberar
77     cambiadas = tuple(sorted(self._bloqueadas ^ nuevas))
78     if cambiadas:
79         self._bloqueadas = nuevas
80         self._version += 1
81     return cambiadas

```

- 1 `transitable` = caminable en el mapa y no bloqueada en esta sesión.
- 2 `vecinos` es la puerta de entrada al grafo: aplica las reglas de `movimiento.py` usando el `transitable` de la sesión.
- 3 `costo_ruta` recorre una ruta y suma sus pesos. Si un paso no es una arista legal, falla. La verificación lo usa para no confiar ciegamente en el costo que informa cada algoritmo.
- 4 `actualizar_obstaculos` valida todo antes de cambiar nada (es atómico), calcula qué celdas cambiaron de verdad con la diferencia simétrica `^` y sube la `version` solo si hubo cambio.

LA IDEA DE MINECRAFT, EN CÓDIGO

El NPC sigue su ruta; aparece un bloque → `sesion.actualizar_obstaculos(bloquear=[celda])`; se vuelve a buscar desde la posición actual. Como `vecinos()` se calcula al momento, la nueva búsqueda ya ve el bloque.

III

Los algoritmos

Cuatro buscadores que comparten el mismo esqueleto y solo difieren en qué celda examinan primero. Más UFDS para las regiones.

-
- 7 El contrato, el catálogo y el observador
 - 8 El esqueleto común y el mapa de ejemplo
 - 9 BFS
 - 10 Dijkstra
 - 11 Voraz primero el mejor
 - 12 A*
 - 13 UFDS
 - 14 Los cuatro, frente a frente

El contrato, el catálogo y el observador

Antes de ver los algoritmos, tres piezas pequeñas que los hacen intercambiables y medibles.

El contrato

Todos los algoritmos tienen la misma firma: reciben el grafo, el inicio, el destino y un observador, y devuelven un `ResultadoBusqueda`. Es el equivalente a una *interfaz* de Java o C#: quien usa un algoritmo no necesita saber cuál es.

src/npc_nav/algorithms/contrato.py

Python · código real del repositorio

```

19 class Grafo(Protocol):
20     def vecinos(self, posicion: Posicion) → Iterable[tuple[Posicion, float]]: ...
21
22
23 class Observador(Protocol):
24     def frontera(self, posicion: Posicion, g: float, h: float,
25                 padre: Posicion | None) → None:
26         """Una celda entra a la frontera o mejora su costo conocido."""
27
28     def expandido(self, posicion: Posicion, g: float, h: float,
29                 padre: Posicion | None) → None:
30         """Una celda sale de la frontera y se procesan sus vecinos."""
31
32
33 class ObservadorNulo:
34     """Observador que ignora todos los eventos."""
35
36     def frontera(self, posicion, g, h, padre):
37         pass
38
39     def expandido(self, posicion, g, h, padre):
40         pass
41
42
43 @dataclass(frozen=True)
44 class ResultadoBusqueda:
45     ruta: tuple[Posicion, ...]
46     """Posiciones desde el inicio hasta el destino; vacía si no hay ruta."""
47     costo: float | None
48     """Suma de pesos de la ruta; None si no hay ruta."""
49
50     @property
51     def encontrada(self) → bool:
52         return bool(self.ruta)
53
54
55 def reconstruir_ruta(padres: dict[Posicion, Posicion], inicio: Posicion,
56                     destino: Posicion) → tuple[Posicion, ...]:
57     """Sigue los punteros `padre` desde el destino hasta el inicio."""
58     ruta = [destino]
59     while ruta[-1] ≠ inicio:
60         ruta.append(padres[ruta[-1]])
61     return tuple(reversed(ruta))

```

1

`Grafo` es un `Protocol`: cualquier objeto con un método `vecinos` sirve. `SesionNavegacion` lo cumple sin heredar de nada.

- 2 `ResultadoBusqueda` guarda la ruta (tupla de posiciones) y su costo. Sin ruta, ambos quedan vacíos (`SIN_RUTA`).
- 3 `reconstruir_ruta` sigue el diccionario `padres` desde el destino hacia atrás y luego invierte la lista. Todos los algoritmos la usan.

El observador



Figura 7.1 · Patrón Observer. El algoritmo avisa sus pasos sin saber quién escucha. La capa de aplicación decide si el oyente cuenta nodos o graba la traza.

El algoritmo lleva consigo un «reportero» y le avisa en dos momentos: `frontera(pos, g, h, padre)` cuando descubre una celda o encuentra un camino más barato hacia ella, y `expandido(pos, g, h, padre)` cuando la saca de pendientes para examinarla. Gracias a eso **el código del algoritmo no tiene cronómetros ni contadores** y se lee igual que su pseudocódigo.

El catálogo

Un diccionario que traduce el texto `"astar"` a la función `astar`, con los datos que muestran el CLI y la web.

src/npc_nav/algorithms/catalogo.py

Python · código real del repositorio

```

28 ALGORITMOS: dict[str, InfoAlgoritmo] = {info.clave: info for info in (
29     InfoAlgoritmo('bfs', 'BFS (anchura)', bfs, False,
30         'Recorridos y búsquedas en grafos', '0(V + E)',
31         'Cormen et al. (2009)'),
32     InfoAlgoritmo('dijkstra', 'Dijkstra', dijkstra, True,
33         'Recorridos y búsquedas en grafos', '0(E log V)',
34         'Dijkstra (1959)'),
35     InfoAlgoritmo('voraz', 'Voraz primero el mejor', voraz, False,
36         'Algoritmos voraces', '0(E log V)',
37         'Russell y Norvig (2021)'),
38     InfoAlgoritmo('astar', 'A*', astar, True,
39         'Algoritmos voraces (heurístico) + búsqueda en grafos', '0(E log V)',
40         'Hart, Nilsson y Raphael (1968)'),
41 )}
42
43
44 def obtener(clave: str) → InfoAlgoritmo:
45     try:
46         return ALGORITMOS[clave]
47     except KeyError:
48         disponibles = ', '.join(ALGORITMOS)
49         raise ValueError(f'Algoritmo desconocido: {clave!r}. Disponibles: {disponibles}') from None

```

El esqueleto común

Los cuatro buscadores repiten el mismo ciclo. Lo único que cambia es el criterio para elegir qué celda examinar después.

1. poner el inicio en la FRONTERA (lista de pendientes) con $g = 0$
2. mientras la frontera no esté vacía:
3. sacar una celda según EL CRITERIO DEL ALGORITMO ← lo único que cambia
4. si es el destino: reconstruir la ruta siguiendo los padres y terminar
5. EXPANDIRLA: para cada vecino (grafo.vecinos):
6. si es nuevo o se llega más barato: anotar su g y su padre,
7. y agregarlo a la frontera
8. si la frontera se vació: no hay ruta

ALGORITMO	SACA PRIMERO...	FRONTERA	¿ÓPTIMO?	COMPLEJIDAD
BFS	la celda más antigua	cola FIFO (<code>deque</code>)	No: minimiza pasos	$O(V + E)$
Dijkstra	la de menor g	montículo (<code>heapq</code>)	Sí	$O(E \log V)$
Voraz	la de menor h	montículo	No	$O(E \log V)$
A*	la de menor $f = g + h$	montículo	Sí	$O(E \log V)$

g es lo que ya se recorrió desde el inicio (real). h es lo que se estima que falta (distancia octil). f es la suma: el costo total estimado de un camino que pase por esa celda.

El mapa de ejemplo

Para ver cada algoritmo celda por celda usamos un mapa pequeño de 9×13 con una **trampa en forma de U** abierta hacia el inicio. El NPC parte de (4, 2) (●) y debe llegar a (4, 11) (★). La línea recta lo mete dentro de la U: un algoritmo que solo mire hacia el destino cae en la trampa.

```

    0 1 2 3 4 5 6 7 8 9 10 11 12
0   . . . . . . . . . . . . .
1   . . . . . . . . . . . . .
2   . . . . . @ @ @ @ . . . .
3   . . . . . . . . @ . . . .
4   . . ● . . . . @ . . ★ . .
5   . . . . . . . . @ . . . .
6   . . . . . @ @ @ @ . . . .
7   . . . . . . . . . . . . .
8   . . . . . . . . . . . . .

```

En las figuras de los capítulos siguientes, cada celda azul lleva un número: el orden en que el algoritmo la expandió. Así se ve con exactitud qué examina primero cada uno.

■ expandida (con su orden) ■ descubierta, pendiente ■ ruta final ■ obstáculo

BFS — búsqueda en anchura

Explora por capas, como una mancha de agua: primero todo lo que está a un paso, luego a dos, y así. No sabe dónde está el destino ni distingue un paso recto de uno diagonal.

Pseudocódigo

```
1. marcar inicio como visitado; encolar inicio
2. mientras la cola no esté vacía:
3.     actual = desencolar (el más antiguo, FIFO)
4.     si actual es el destino: devolver la ruta reconstruida
5.     para cada vecino v de actual no visitado:
6.         marcar v como visitado; padre[v] = actual
7.         encolar v
8. devolver SIN_RUTA
```

Código

src/npc_nav/algorithms/bfs.py

Python · código real del repositorio

```
32 def bfs(grafo, inicio, destino, observador=NULO) → ResultadoBusqueda:
33     g = {inicio: 0.0} # 1.
34     padre = {}
35     cola = deque([inicio])
36     observador.frontera(inicio, 0.0, 0.0, None)
37
38     while cola: # 2.
39         actual = cola.popleft() # 3. FIFO
40         observador.expandido(actual, g[actual], 0.0, padre.get(actual))
41
42         if actual == destino: # 4.
43             return ResultadoBusqueda(reconstruir_ruta(padre, inicio, destino), g[actual])
44
45         for vecino, peso in grafo.vecinos(actual): # 5.
46             if vecino in g:
47                 continue # ya visitado
48             g[vecino] = g[actual] + peso # 6.
49             padre[vecino] = actual
50             cola.append(vecino) # 7.
51             observador.frontera(vecino, g[vecino], 0.0, actual)
52
53     return SIN_RUTA # 8.
```

- 1 `deque` es una cola doble: `append` agrega al final y `popleft` saca del inicio en $O(1)$. Eso produce el orden FIFO.
- 2 El diccionario `g` cumple dos papeles: marca las celdas visitadas (`if vecino in g`) y guarda el costo real del camino encontrado, solo para informarlo.
- 3 Una celda se marca como visitada **al descubrirla**, no al expandirla. Por eso nunca entra dos veces a la cola, y por eso nunca corrige un camino por otro más barato.

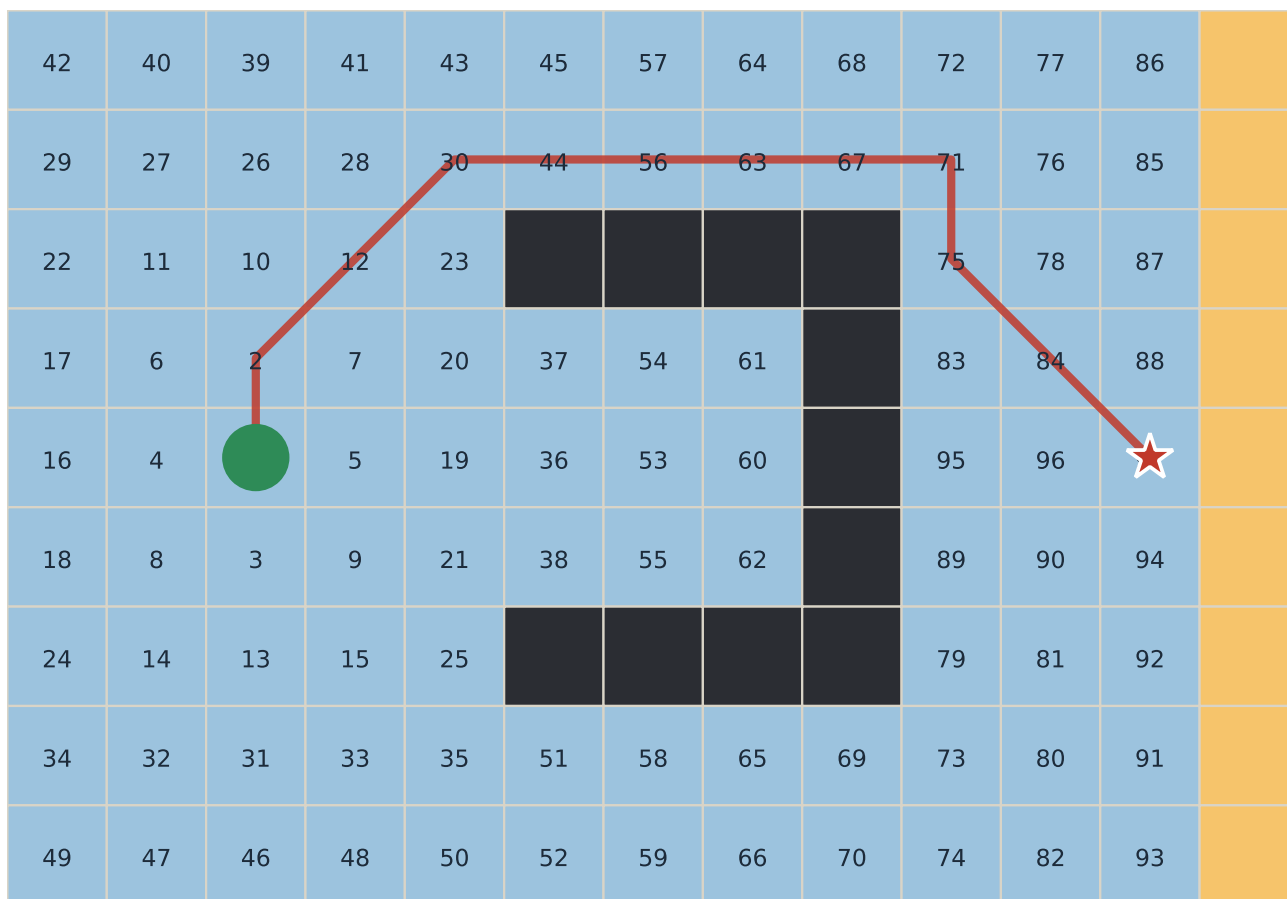


Figura 9.1 · BFS en la trampa en U. Los números crecen en anillos alrededor del inicio, sin preferir ninguna dirección: la celda 97 es el destino. Expandió 97 celdas y encontró un camino de costo 12.66.

Los primeros eventos

Así se ve la traza que BFS le entrega al observador al comenzar. Primero descubre (*frontera*) los 8 vecinos del inicio y recién después empieza a expandirlos en el orden en que entraron:

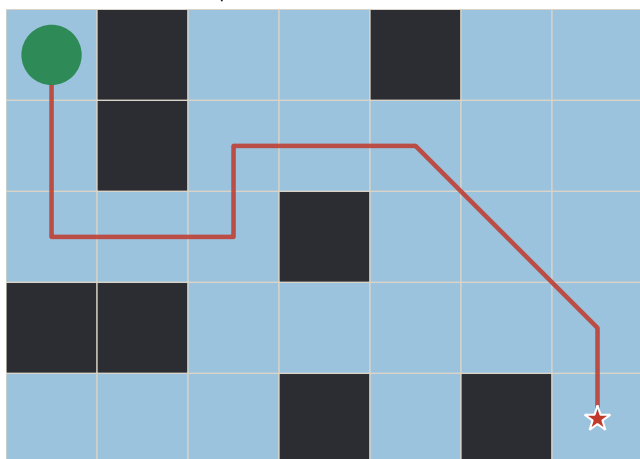
#	AVISO	CELDA	G
1	frontera	(4, 2)	0.000
2	expandido	(4, 2)	0.000
3	frontera	(3, 2)	1.000
4	frontera	(5, 2)	1.000
5	frontera	(4, 1)	1.000
6	frontera	(4, 3)	1.000
7	frontera	(3, 1)	1.414
8	frontera	(3, 3)	1.414
9	frontera	(5, 1)	1.414
10	frontera	(5, 3)	1.414

#	AVISO	CELDA	G
11	expandido	(3, 2)	1.000
12	frontera	(2, 2)	2.000
13	frontera	(2, 1)	2.414
14	frontera	(2, 3)	2.414

¿Por qué no es óptimo?

En la trampa en U, BFS acertó con el camino óptimo por la forma del mapa. Pero BFS minimiza el número de pasos, no la distancia. Cuando hay varios caminos con los mismos pasos, puede elegir uno con más diagonales, que cuestan 1.414 cada una:

BFS (anchura) — 10 pasos · costo 10.83



A* — 10 pasos · costo 10.00

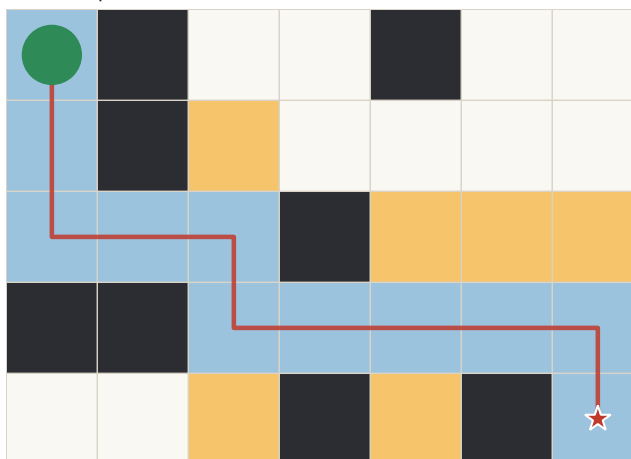


Figura 9.2 · Mismo número de pasos, distinto costo. Ambos caminos tienen 10 pasos. BFS elige uno con más diagonales y cuesta 10.83; A* elige el de costo 10.00. Este mapa es una de las pruebas automáticas del proyecto.

ROL EN EL PROYECTO

BFS es la **línea base**: el recorrido más simple del curso. Demuestra por qué un grafo con pesos necesita algoritmos que los tengan en cuenta.

Dijkstra

Igual que BFS, pero siempre examina la celda con menor costo acumulado g . Con pesos no negativos, eso garantiza el camino óptimo.

Pseudocódigo

```

1.  $g[\text{inicio}] = 0$ ; insertar inicio en la frontera con prioridad 0
2. mientras la frontera no esté vacía:
3.      $\text{actual} = \text{extraer el nodo de menor } g$ 
4.     si  $\text{actual}$  es el destino: devolver la ruta reconstruida
5.     marcar  $\text{actual}$  como cerrado
6.     para cada vecino  $v$  de  $\text{actual}$  con peso  $w$ :
7.         si  $g[\text{actual}] + w < g[v]$ :
8.              $g[v] = g[\text{actual}] + w$ ;  $\text{padre}[v] = \text{actual}$ 
9.             insertar  $v$  en la frontera con prioridad  $g[v]$ 
10. devolver SIN_RUTA

```

Código

src/npc_nav/algorithms/dijkstra.py

Python · código real del repositorio

```

34 def dijkstra(grafo, inicio, destino, observador=NULO) → ResultadoBusqueda:
35     orden = count()
36
37     g = {inicio: 0.0} # 1.
38     padre = {}
39     cerrados = set()
40     frontera = [(0.0, next(orden), inicio)] # (g, orden, nodo)
41     observador.frontera(inicio, 0.0, 0.0, None)
42
43     while frontera: # 2.
44         _, _, actual = heappop(frontera) # 3. menor g
45         if actual in cerrados:
46             continue # copia antigua en la cola
47         observador.expandido(actual, g[actual], 0.0, padre.get(actual))
48
49         if actual == destino: # 4.
50             return ResultadoBusqueda(reconstruir_ruta(padre, inicio, destino), g[actual])
51         cerrados.add(actual) # 5.
52
53         for vecino, peso in grafo.vecinos(actual): # 6.
54             if vecino in cerrados:
55                 continue
56             nuevo_g = g[actual] + peso
57             if nuevo_g < g.get(vecino, inf): # 7. relajación
58                 g[vecino] = nuevo_g # 8.
59                 padre[vecino] = actual
60                 heappush(frontera, (nuevo_g, next(orden), vecino)) # 9.
61                 observador.frontera(vecino, nuevo_g, 0.0, actual)
62
63     return SIN_RUTA # 10.

```

- 1 La frontera es un **montículo mínimo** (`heapq`): `heappop` entrega siempre la tupla más pequeña en $O(\log n)$. La tupla empieza con `g`, así que sale la celda más cercana al inicio.
- 2 `next(orden)` es un contador creciente. Desempata dos celdas con el mismo `g` por orden de llegada, y evita que Python compare tuplas de posiciones.
- 3 **Relajación** (paso 7): si el camino por `actual` es más barato que el mejor conocido hacia `vecino`, se actualizan su `g` y su `padre`.
- 4 `heapq` no permite cambiar la prioridad de un elemento. Cuando una celda mejora, se inserta otra copia con el costo nuevo; al salir la copia vieja, `if actual in cerrados: continue` la descarta.

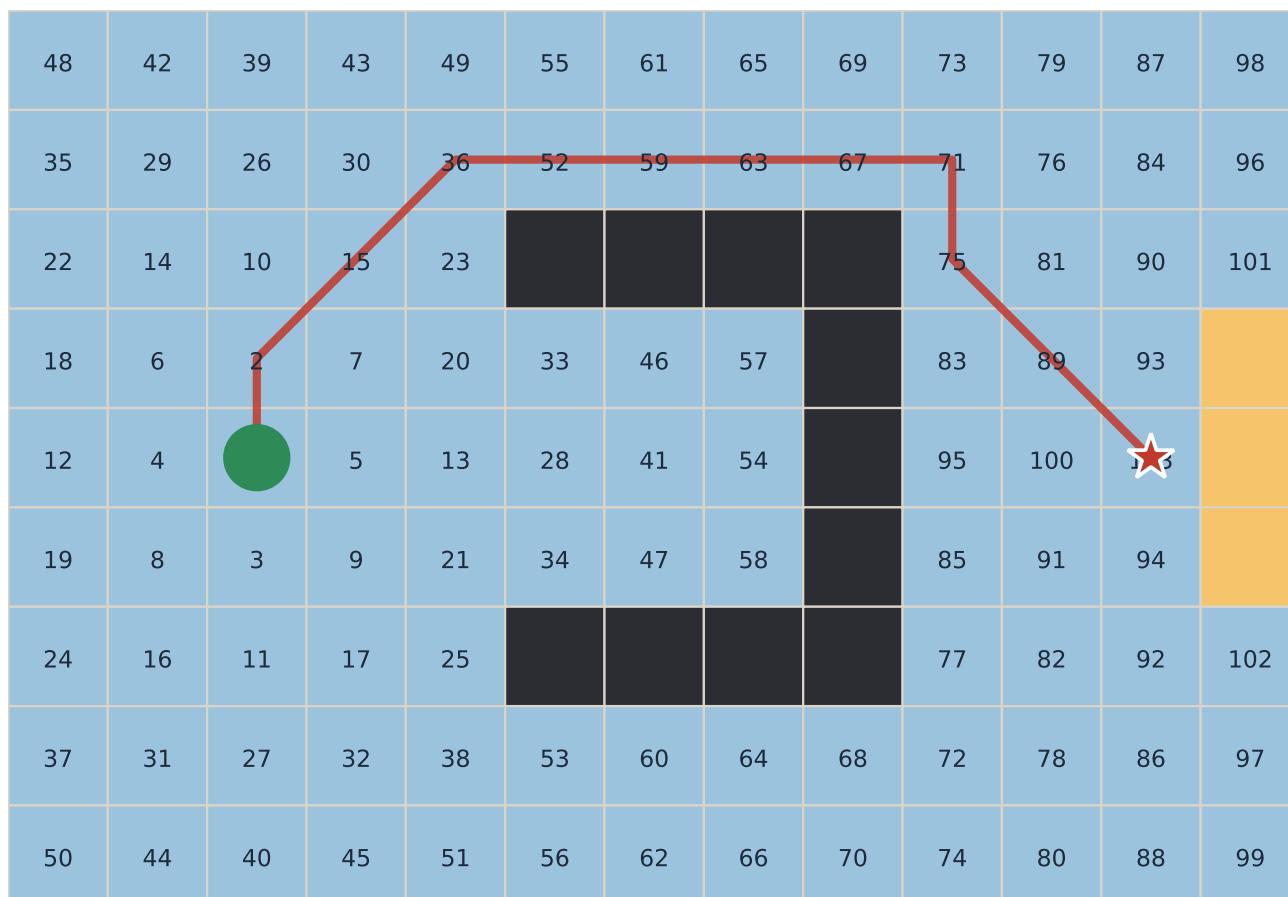


Figura 10.1 · Dijkstra en la trampa en U. Crece en círculos de costo alrededor del inicio: primero las celdas rectas (costo 1), después las diagonales (1.414). Expandió 103 celdas para un camino de costo 12.66, el óptimo.

Las primeras expansiones

El orden sale estrictamente por `g`: primero el inicio (0), luego sus cuatro vecinos rectos (1) y después los diagonales (1.414). Todavía no se acerca al destino: examina igual la celda `(4, 1)`, que está detrás del inicio, que la `(4, 3)`, que está delante.

#	CELDA	G (PRIORIDAD)
1	<code>(4, 2)</code>	0.000
2	<code>(3, 2)</code>	1.000
3	<code>(5, 2)</code>	1.000

#	CELDA	G (PRIORIDAD)
4	(4, 1)	1.000
5	(4, 3)	1.000
6	(3, 1)	1.414
7	(3, 3)	1.414
8	(5, 1)	1.414

SU DEBILIDAD

Dijkstra no sabe dónde está el destino. Explora igual hacia atrás que hacia adelante, y por eso expande casi lo mismo que BFS: 6 537 celdas en el caso 375 del mapa real.

Voraz primero el mejor

Examina siempre la celda que parece más cercana al destino (menor h) e ignora cuánto camino lleva recorrido. Es muy rápido, pero se deja engañar.

Pseudocódigo

```
1. marcar inicio como visitado; insertarlo en la frontera con prioridad h(inicio)
2. mientras la frontera no esté vacía:
3.     actual = extraer el nodo de menor h (el que PARECE más cercano al destino)
4.     si actual es el destino: devolver la ruta reconstruida
5.     para cada vecino v de actual no visitado:
6.         marcar v como visitado; padre[v] = actual
7.         insertar v en la frontera con prioridad h(v)
8. devolver SIN_RUTA
```

Código

src/npc_nav/algorithms/voraz.py

Python · código real del repositorio

```
34 def voraz(grafo, inicio, destino, observador=NULO) → ResultadoBusqueda:
35     h = lambda posicion: distancia_octil(posicion, destino)
36     orden = count()
37
38     g = {inicio: 0.0} # 1.
39     padre = {}
40     frontera = [(h(inicio), next(orden), inicio)] # (h, orden, nodo)
41     observador.frontera(inicio, 0.0, h(inicio), None)
42
43     while frontera: # 2.
44         h_actual, _, actual = heappop(frontera) # 3. menor h
45         observador.expandido(actual, g[actual], h_actual, padre.get(actual))
46
47         if actual == destino: # 4.
48             return ResultadoBusqueda(reconstruir_ruta(padre, inicio, destino), g[actual])
49
50         for vecino, peso in grafo.vecinos(actual): # 5.
51             if vecino in g:
52                 continue # ya visitado
53             g[vecino] = g[actual] + peso # 6.
54             padre[vecino] = actual
55             h_vecino = h(vecino)
56             heappush(frontera, (h_vecino, next(orden), vecino)) # 7.
57             observador.frontera(vecino, g[vecino], h_vecino, actual)
58
59     return SIN_RUTA # 8.
```

- 1 La prioridad del montículo es `h`, la distancia octil al destino. `g` se acumula solo para informar el costo de la ruta; no influye en ninguna decisión.

- Como BFS, marca las celdas al descubrirlas y nunca corrige un camino. Una vez que llegó a una celda por un rodeo, ese rodeo queda en la ruta final.
- Es una **estrategia voraz**: toma la decisión localmente mejor en cada paso y no la reconsidera. Es la técnica «Algoritmos voraces» del curso.

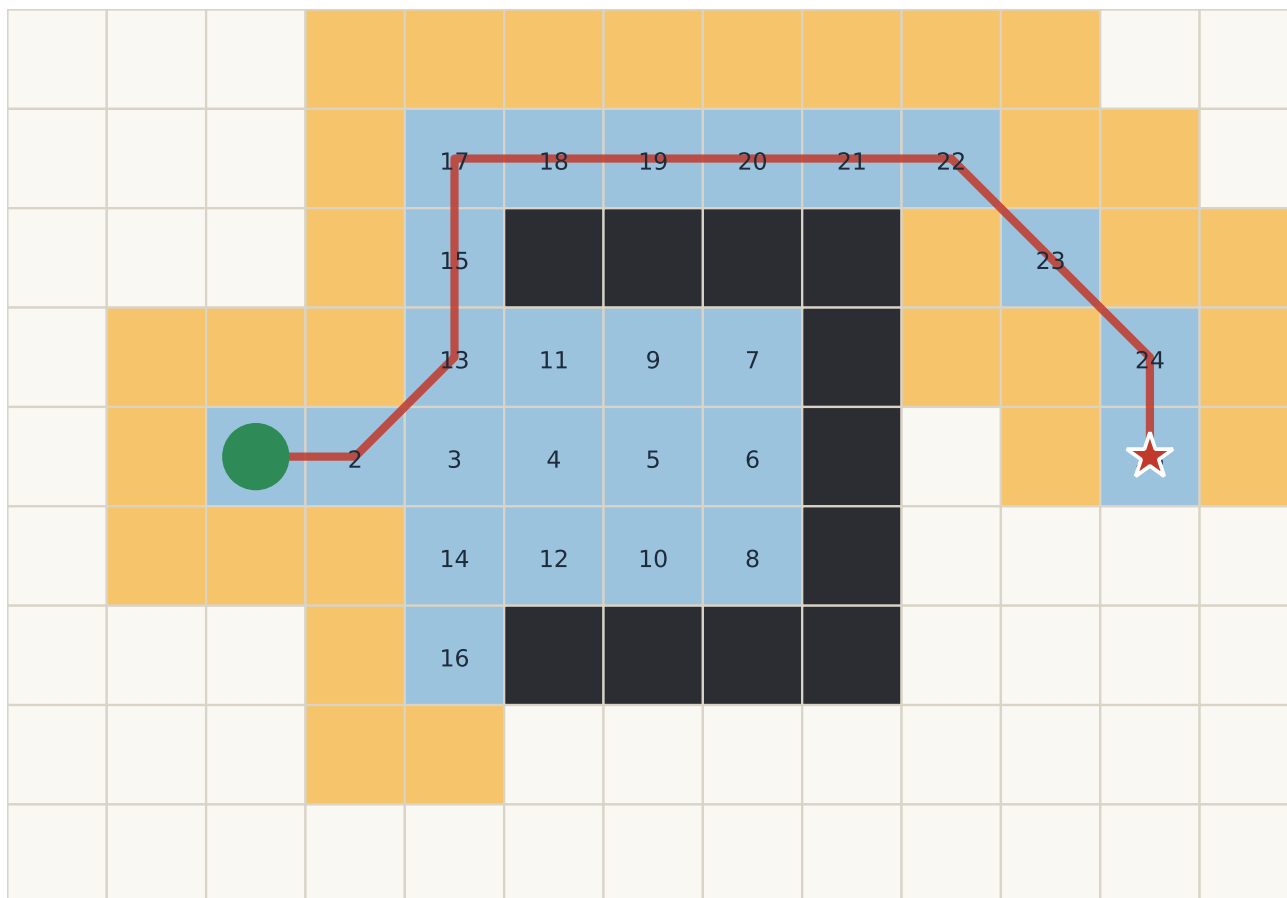


Figura 11.1 · El voraz cae en la trampa. Las celdas 1 a 6 van en línea recta hacia el destino hasta tocar el fondo de la U. Después revisa las celdas pegadas al muro, porque son las que «parecen» más cercanas, y solo entonces rodea la U. Solo expandió 25 celdas, pero su camino cuesta 13.24 frente al óptimo de 12.66.

Las primeras expansiones

Mira la columna **h**: siempre sale la celda con la menor estimación, aunque eso signifique meterse en un callejón.

#	CELDA	H (PRIORIDAD)	G (SOLO INFORMATICA)
1	(4, 2)	9.000	0.000
2	(4, 3)	8.000	1.000
3	(4, 4)	7.000	2.000
4	(4, 5)	6.000	3.000
5	(4, 6)	5.000	4.000
6	(4, 7)	4.000	5.000

#	CELDA	H (PRIORIDAD)	G (SOLO INFORMA)
7	(3, 7)	4.414	5.414
8	(5, 7)	4.414	5.414
9	(3, 6)	5.414	4.414
10	(5, 6)	5.414	4.414

ROL EN EL PROYECTO

El voraz muestra el otro extremo: la velocidad sin garantía de calidad. En el mapa real expande 332 celdas (20 veces menos que Dijkstra), pero su ruta es un 25 % más larga. En `brc100d`, caso 1160, llega a ser 293 unidades más larga que la óptima.

A*

Combina las dos ideas: examina la celda con menor $f = g + h$, lo recorrido más lo que falta. g evita los rodeos del voraz y h evita la exploración a ciegas de Dijkstra.

Pseudocódigo

```

1. g[inicio] = 0; insertar inicio en la frontera con f = h(inicio)
2. mientras la frontera no esté vacía:
3.     actual = extraer el nodo de menor f = g + h
4.     si actual es el destino: devolver la ruta reconstruida
5.     marcar actual como cerrado
6.     para cada vecino v de actual con peso w:
7.         si g[actual] + w < g[v]:
8.             g[v] = g[actual] + w; padre[v] = actual
9.             insertar v en la frontera con f = g[v] + h(v)
10. devolver SIN_RUTA

```

Código

src/npc_nav/algorithms/astar.py

Python · código real del repositorio

```

38 def astar(grafo, inicio, destino, observador=NULO) → ResultadoBusqueda:
39     h = lambda posicion: distancia_octil(posicion, destino)
40     orden = count()
41
42     # 1. el inicio entra a la frontera con g = 0
43     g = {inicio: 0.0}
44     padre = {}
45     cerrados = set()
46     frontera = [(h(inicio), h(inicio), next(orden), inicio)] # (f, h, orden, nodo)
47     observador.frontera(inicio, 0.0, h(inicio), None)
48
49     while frontera: # 2.
50         _, h_actual, _, actual = heappop(frontera) # 3. menor f
51         if actual in cerrados:
52             continue # copia antigua en la cola: ya se procesó con mejor g
53         observador.expandido(actual, g[actual], h_actual, padre.get(actual))
54
55         if actual == destino: # 4.
56             return ResultadoBusqueda(reconstruir_ruta(padre, inicio, destino), g[actual])
57         cerrados.add(actual) # 5.
58
59         for vecino, peso in grafo.vecinos(actual): # 6.
60             if vecino in cerrados:
61                 continue
62             nuevo_g = g[actual] + peso
63             if nuevo_g < g.get(vecino, inf): # 7. relajación
64                 g[vecino] = nuevo_g # 8.
65                 padre[vecino] = actual
66                 h_vecino = h(vecino)
67                 heappush(frontera, (nuevo_g + h_vecino, h_vecino, next(orden), vecino)) # 9.

```

```

68         observador.frontera(vecino, nuevo_g, h_vecino, actual)
69
70     return SIN_RUTA # 10.

```

- 1 `h` es una función pequeña (`lambda`) que fija el destino: `h(celda)` = distancia octil de esa celda al destino.
- 2 La tupla del montículo es `(f, h, orden, nodo)` . Sale la menor `f` ; a igual `f` , la menor `h` (la que está más cerca del destino); y si todavía empatan, la que entró antes. Por eso la misma consulta da siempre la misma ruta y los mismos eventos.
- 3 Pasos 7–9: la misma relajación de Dijkstra, pero la prioridad es `nuevo_g + h_vecino` .
- 4 Como la heurística es consistente, una celda cerrada ya tiene su costo óptimo y no hace falta reabrirla (`if vecino in cerrados: continue`).

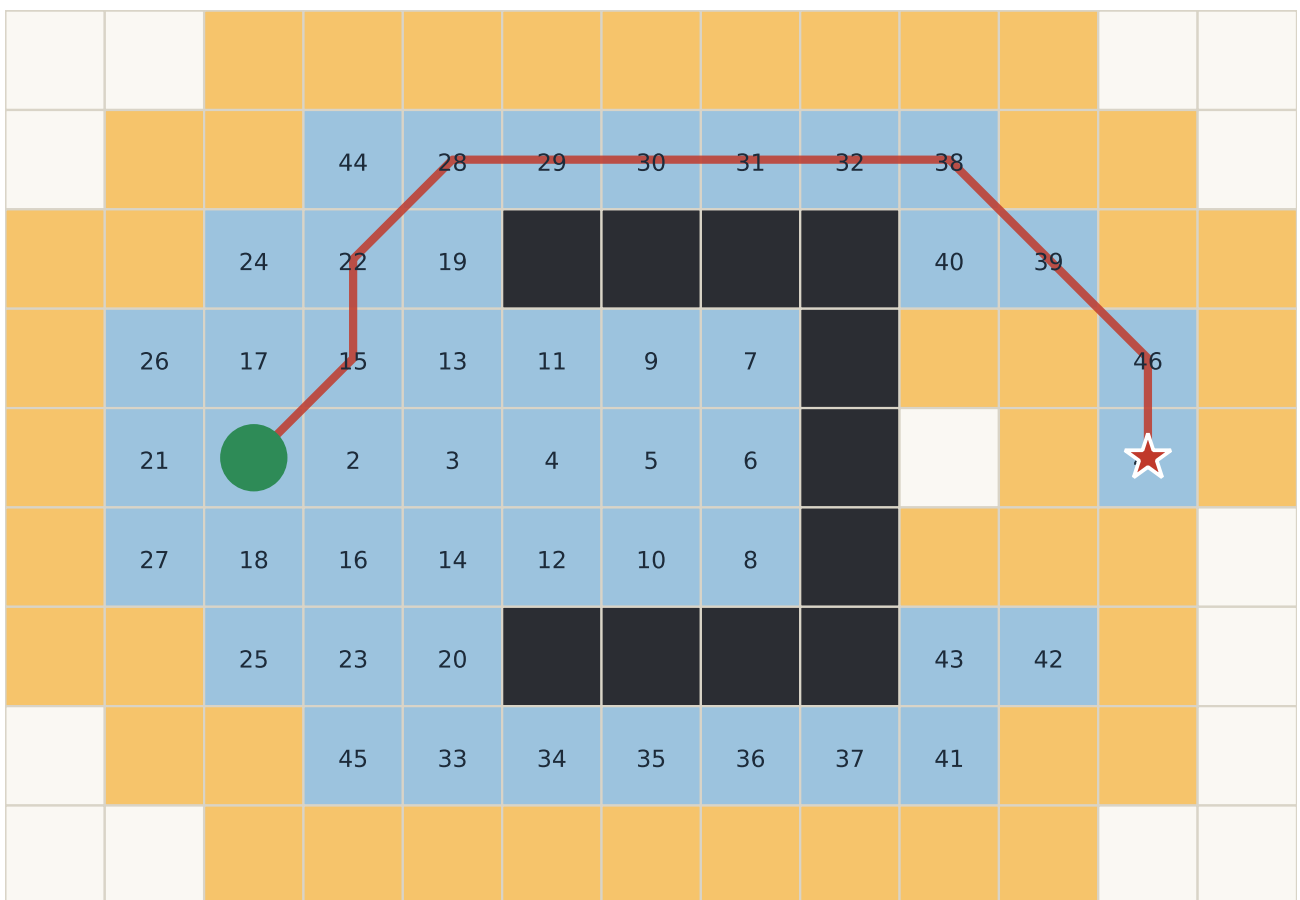


Figura 12.1 · A* en la trampa en U. Las celdas 1 a 6 también entran a la U en línea recta ($f = 9$, lo más prometedor). Al chocar, f sube a 9.83 y sigue subiendo mientras insiste en el fondo; como g cuenta lo recorrido, los caminos que rodean la U terminan siendo más baratos y A* los toma. Expandió 47 celdas, menos de la mitad que Dijkstra, con el mismo costo óptimo de 12.66.

A* paso a paso

Las primeras expansiones en la trampa en U. Mientras el camino recto es posible, f se mantiene en 9: cada paso suma 1 a g y resta 1 a h . En la fila 7 choca con la U: la mejor f disponible sube a 9.83 y A* empieza a revisar las celdas vecinas del corredor. La columna **padre** es lo que después permite reconstruir la ruta.

#	CELDA	G	H	F = G + H	PADRE
1	(4, 2)	0.000	9.000	9.000	—
2	(4, 3)	1.000	8.000	9.000	(4, 2)
3	(4, 4)	2.000	7.000	9.000	(4, 3)
4	(4, 5)	3.000	6.000	9.000	(4, 4)
5	(4, 6)	4.000	5.000	9.000	(4, 5)
6	(4, 7)	5.000	4.000	9.000	(4, 6)
7	(3, 7)	5.414	4.414	9.828	(4, 6)
8	(5, 7)	5.414	4.414	9.828	(4, 6)
9	(3, 6)	4.414	5.414	9.828	(4, 5)
10	(5, 6)	4.414	5.414	9.828	(4, 5)
11	(3, 5)	3.414	6.414	9.828	(4, 4)
12	(5, 5)	3.414	6.414	9.828	(4, 4)

POR QUÉ A* GARANTIZA EL ÓPTIMO

Cuando A* saca el destino de la frontera, su **f** es su costo real (ahí $h = 0$). Cualquier otra celda pendiente tiene una **f** mayor o igual, y como h nunca sobreestima, ningún camino que pase por ella puede costar menos. Esta es la demostración de Hart, Nilsson y Raphael (1968).

UFDS — regiones conectadas

No busca rutas: responde en tiempo casi constante si dos celdas están en la misma región del mapa. En brc201d detecta 167 regiones aisladas.

Pseudocódigo

```

encontrar(x):
1. subir por los padres hasta la raíz (el representante del conjunto)
2. compresión de caminos: colgar cada nodo recorrido directamente de la raíz

unir(a, b):
3. ra = encontrar(a); rb = encontrar(b); si son iguales, ya están unidos
4. unión por rango: colgar el árbol más bajo del más alto

regiones_conectadas(grafo):
5. crear un conjunto por vértice
6. para cada arista (u, v): unir(u, v)
7. dos vértices están en la misma región si y solo si tienen la misma raíz

```

Código

src/npc_nav/algorithms/ufds.py

Python · código real del repositorio

```

46     def encontrar(self, x):
47         raiz = x
48         while self.padre[raiz] != raiz:           # 1.
49             raiz = self.padre[raiz]
50         while self.padre[x] != raiz:             # 2. compresión
51             self.padre[x], x = raiz, self.padre[x]
52         return raiz
53
54     def unir(self, a, b) → bool:
55         ra, rb = self.encontrar(a), self.encontrar(b)   # 3.
56         if ra == rb:
57             return False
58         if self.rango[ra] < self.rango[rb]:           # 4. unión por rango
59             ra, rb = rb, ra
60         self.padre[rb] = ra
61         self.tamano[ra] += self.tamano[rb]
62         if self.rango[ra] == self.rango[rb]:
63             self.rango[ra] += 1
64         self.num_conjuntos -= 1
65         return True
66
67     def regiones_conectadas(grafo: dict) → tuple[UFDS, list[int]]:
68         """Recibe una lista de adyacencia {nodo: [vecinos]}.
69
70         Devuelve la estructura UFDS y los tamaños de las regiones, de mayor a menor.
71         """
72         ufds = UFDS()
73         for nodo in grafo:                             # 5.
74             ufds.agregar(nodo)
75         for nodo, vecinos in grafo.items():             # 6.

```

```

80         for vecino in vecinos:
81             udfs.unir(nodo, vecino)
82     tamanos = sorted((ufds.tamano[nodo] for nodo in grafo # 7.
83                     if udfs.encontrar(nodo) == nodo), reverse=True)
84     return udfs, tamanos

```

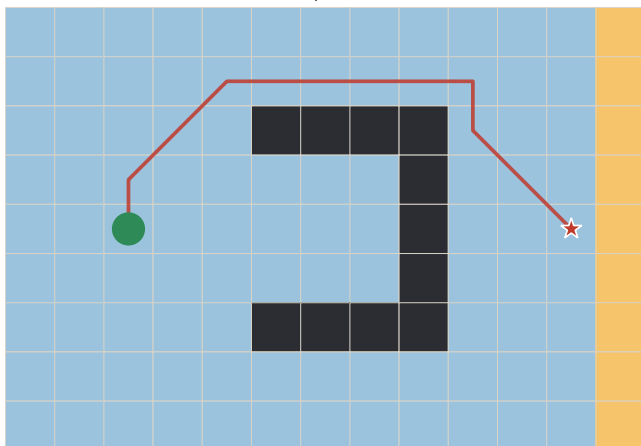
- 1 Cada conjunto es un árbol; su raíz es el **representante**. Dos celdas están en la misma región si y solo si `encontrar` devuelve la misma raíz.
- 2 **Compresión de caminos:** después de subir hasta la raíz, cada nodo recorrido se cuelga directamente de ella. La próxima consulta es inmediata.
- 3 **Unión por rango:** el árbol más bajo se cuelga del más alto, así los árboles nunca crecen mucho.
- 4 Con ambas técnicas, cada operación cuesta $O(\alpha(V))$ amortizado. α (inversa de Ackermann) es menor que 5 para cualquier tamaño imaginable: en la práctica, constante.

En el proyecto se usa para validar el dataset (toda misión del solucionario debe tener inicio y destino en la misma región) y para la figura de regiones del informe TB1. Describe el mapa **original**: si se bloquean celdas, la conectividad debe volver a comprobarse.

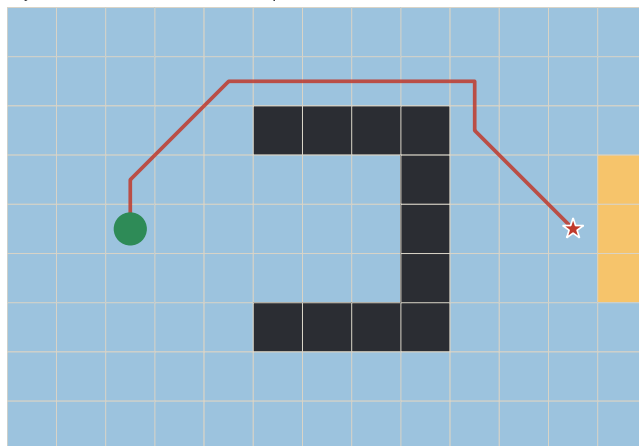
Los cuatro, frente a frente

La misma misión, el mismo mapa y el mismo mecanismo de medición.
La diferencia entre ellos es únicamente el criterio para elegir la siguiente celda.

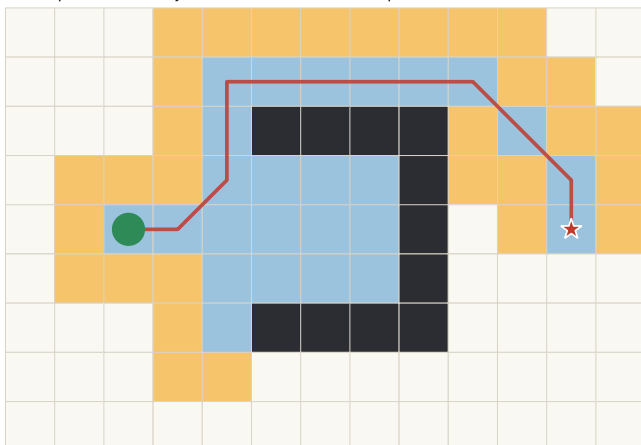
BFS (anchura) — costo 12.66 · 97 expandidas



Dijkstra — costo 12.66 · 103 expandidas



Voraz primero el mejor — costo 13.24 · 25 expandidas



A* — costo 12.66 · 47 expandidas

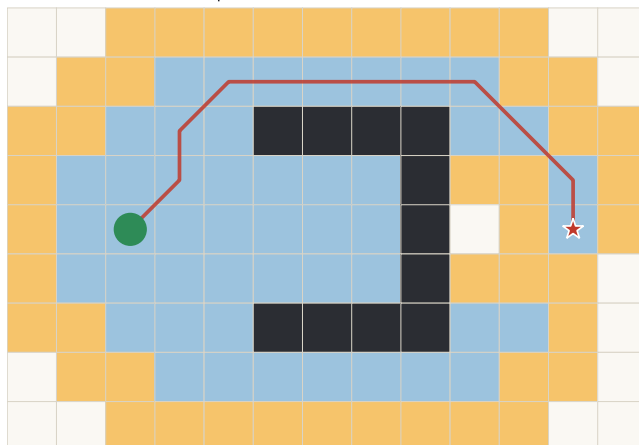


Figura 14.1 · Trampa en U. Sin números, para comparar la forma de la exploración: BFS y Dijkstra se expanden en todas direcciones, el voraz va directo y cae en la U, A* se concentra en el corredor útil.

Los números

ALGORITMO	U: COSTO	U: EXPANDIDAS	DEN011D-375: COSTO	EXPANDIDAS	DESCUBIERTAS	MÁX. FRONTERA
BFS	12.66	97	148.25	6 491	6 572	111
Dijkstra	12.66	103	148.25	6 537	6 643	116
Voraz	13.24	25	185.17	332	689	358
A*	12.66	47	148.25	2 080	2 226	247

Óptimos: 12.66 en la trampa en U y 148.25 en el caso 375. «máx. frontera» es la memoria que necesitó cada algoritmo: el voraz y A* guardan más celdas pendientes porque exploran de forma dirigida y su frontera es ancha.

Qué concluir

PREGUNTA	RESPUESTA DEL EXPERIMENTO
¿Quién garantiza el camino más corto?	Dijkstra y A*. BFS solo cuando todos los pasos pesan lo mismo; el voraz nunca.
¿Quién trabaja menos?	El voraz, pero pagando con la calidad de la ruta.
¿Quién da el mejor equilibrio?	A*: ruta óptima con 3 veces menos celdas que Dijkstra en el mapa real.
¿Qué falta medir?	Qué pasa cuando aparecen obstáculos: A* recalcula todo desde cero. D* Lite, en el hito 2, reutiliza el trabajo anterior.

IV

Ejecutar, medir y mostrar

Cómo un comando en la terminal recorre todas las capas y termina en una tabla, una imagen o un archivo de evidencia.

15 La ejecución y las métricas: busqueda.py

16 Recorrido de npc-nav comparar

17 Cómo se dibuja la imagen

18 La verificación contra el solucionario

La ejecución y las métricas

`ejecutar_busqueda` es el director de orquesta: valida la consulta, elige el algoritmo, le entrega un observador, toma el tiempo y devuelve todo empaquetado.

Los dos observadores

src/npc_nav/application/busqueda.py

Python · código real del repositorio

```

55 class _Contador:
56     def __init__(self):
57         self.expandidos = 0
58         self.descubiertos = set()
59         self.pendientes = set()
60         self.max_frontera = 0
61
62     def frontera(self, posicion, g, h, padre):
63         self.descubiertos.add(posicion)
64         self.pendientes.add(posicion)
65         self.max_frontera = max(self.max_frontera, len(self.pendientes))
66
67     def expandido(self, posicion, g, h, padre):
68         self.expandidos += 1
69         self.pendientes.discard(posicion)
70
71
72 class _Grabador:
73     def __init__(self):
74         self.eventos = []
75
76     def frontera(self, posicion, g, h, padre):
77         self.eventos.append(EventoBusqueda('frontera', posicion, g, h, padre))
78
79     def expandido(self, posicion, g, h, padre):
80         self.eventos.append(EventoBusqueda('expandido', posicion, g, h, padre))

```

- 1 `_Contador` cuenta expansiones, guarda el conjunto de celdas descubiertas y lleva la cuenta de las pendientes (`pendientes`) para saber el tamaño máximo que tuvo la frontera.
- 2 `_Grabador` solo anota cada evento en una lista, en orden. Esa lista es la **traza** que dibujan la imagen y el dashboard.

`ejecutar_busqueda`

src/npc_nav/application/busqueda.py

Python · código real del repositorio

```

83 def ejecutar_busqueda(sesion: SesionNavegacion, algoritmo: str, inicio: Posicion,
84                       destino: Posicion, *, con_traza: bool = False) → Ejecucion:
85     """Ejecuta `algoritmo` sobre el estado actual de la sesión.
86
87     Rechaza extremos no transitables con ValueError. Un destino libre pero
88     inaccesible produce estado 'sin_ruta'.
89     """

```

```

90     info = obtener(algoritmo)
91     sesion.validar_consulta(inicio, destino)
92     version = sesion.version
93
94     contador = _Contador()
95     reloj = perf_counter_ns()
96     resultado = info.funcion(sesion, inicio, destino, contador)
97     tiempo_ms = (perf_counter_ns() - reloj) / 1_000_000
98
99     traza = ()
100     if con_traza:
101         grabador = _Grabador()
102         info.funcion(sesion, inicio, destino, grabador)
103         traza = tuple(grabador.eventos)
104
105     if sesion.version != version:
106         raise RuntimeError('El mapa cambió durante la búsqueda; vuelva a calcular la ruta')
107     return Ejecucion(
108         algoritmo=info.clave,
109         estado='encontrada' if resultado.encontrada else 'sin_ruta',
110         ruta=resultado.ruta, costo=resultado.costo, version_mapa=version,
111         metricas=MetricasBusqueda(contador.expandidos, len(contador.descubiertos),
112                                   contador.max_frontera, tiempo_ms),
113         traza=traza,
114     )
115
116 ...
117 def comparar_algoritmos(sesion: SesionNavegacion, algoritmos: list[str], inicio: Posicion,
118                         destino: Posicion, *, con_traza: bool = False) → list[Ejecucion]:
119     """Ejecuta varios algoritmos sobre la misma sesión, consulta y estado del mapa."""
120     return [ejecutar_busqueda(sesion, clave, inicio, destino, con_traza=con_traza)
121             for clave in algoritmos]

```

- 1 `obtener(algoritmo)` busca en el catálogo; un nombre desconocido lanza un error con la lista de disponibles.
- 2 El cronómetro (`perf_counter_ns`) mide solo la ejecución con el `_Contador` .
- 3 Si se pide traza, el algoritmo se ejecuta **otra vez** con el `_Grabador` . Como es determinista, produce exactamente los mismos pasos, y el tiempo informado nunca incluye el trabajo de grabar.
- 4 Si la versión de la sesión cambió durante el cálculo (alguien agregó un obstáculo), el resultado se descarta.
- 5 `comparar_algoritmos` solo repite lo anterior para cada algoritmo, sobre la misma sesión y la misma misión. Eso es lo que hace justa la comparación.

MÉTRICA	CÓMO SE CUENTA	QUÉ MIDE	EN LA FIGURA
<code>nodos_expandidos</code>	Un aviso <code>expandido</code> por celda	Trabajo del algoritmo	Azul
<code>nodos_descubiertos</code>	Celdas distintas con aviso <code>frontera</code>	Cuánto llegó a ver	Azul + amarillo
<code>max_frontera</code>	Máximo de pendientes a la vez	Memoria necesaria	Borde amarillo en su punto mayor
<code>tiempo_ms</code>	Milisegundos, sin traza ni archivos	Velocidad en esa máquina	—


```
den011d caso 375: óptimo publicado 148.2548
algoritmo      costo  expandidos  descubiertos  tiempo ms
bfs            148.2548    6491      6572        36.08
dijkstra       148.2548    6537      6643        35.69
voraz          185.1665     332       689         2.06
astar          148.2548    2080      2226        13.55
```

Los tiempos cambian en cada ejecución y de una máquina a otra; los costos y las celdas expandidas son siempre los mismos.

Cómo se dibuja la imagen

La figura 1.1 sigue exactamente el mismo recorrido del capítulo anterior, con una diferencia: pide `con_traza=True` y convierte los eventos en colores.

tools/visualizar_busqueda.py

Python · código real del repositorio

```

33 def pintar(ax, mapa, caso, ejecucion, nombre):
34     matriz = [[LIBRE if celda in '.G' else OBSTACULO for celda in fila] for fila in mapa.celdas]
35     for evento in ejecucion.traza:                                # primero la frontera...
36         if evento.tipo == 'frontera':
37             fila, columna = evento.posicion
38             matriz[fila][columna] = FRONTERA
39     for evento in ejecucion.traza:                                # ...y encima lo expandido
40         if evento.tipo == 'expandido':
41             fila, columna = evento.posicion
42             matriz[fila][columna] = EXPANDIDO
43     ax.imshow(matriz, cmap=COLORES, vmin=0, vmax=3, interpolation='nearest')
44     if ejecucion.ruta:
45         ax.plot([c for _, c in ejecucion.ruta], [f for f, _ in ejecucion.ruta],
46               color='#c0392b', linewidth=2.2)
47     ax.plot(caso.inicio[1], caso.inicio[0], 'o', color='#27ae60', markersize=10, mec='white')
48     ax.plot(caso.destino[1], caso.destino[0], '*', color='#c0392b', markersize=16, mec='white')
49     costo = f'{ejecucion.costo:.1f}' if ejecucion.costo is not None else 'sin ruta'
50     ax.set_title(f'{nombre} - costo {costo} . ')
51     f'{ejecucion.metricas.nodos_expandidos:}, {nodos expandidos}', fontsize=13)
52     ax.set_xticks([])
53     ax.set_yticks([])

```

- 1 Arma una matriz del tamaño del mapa: 0 para suelo y 1 para obstáculo.
- 2 Recorre la traza y marca con 2 (amarillo) cada celda que tuvo un aviso `frontera`.
- 3 Recorre de nuevo y marca con 3 (azul) cada celda `expandido`, encima del amarillo. El amarillo que queda visible es lo que seguía pendiente al terminar.
- 4 `imshow` pinta la matriz con la paleta de cuatro colores; `plot` dibuja la ruta en rojo y los marcadores de inicio y destino.

Para regenerarla

```

cd engine
pip install -e ".[figures]"                # una vez: instala matplotlib
python tools/visualizar_busqueda.py den011d 375    # los 4 algoritmos
python tools/visualizar_busqueda.py brc100d 1160 --algoritmos voraz,astar

```

LO QUE HARÁ EL DASHBOARD

Lo mismo, pero animado: la web recibirá la traza desde la API y pintará los eventos uno por uno en un Canvas, con controles de reproducción. La lógica de búsqueda seguirá solo en el motor.

La verificación contra el solucionario

Para demostrar que A* está bien implementado, se ejecuta en las 4 200 misiones y se compara cada resultado con el óptimo publicado. Coinciden las 4 200.

src/npc_nav/application/verificacion.py

Python · código real del repositorio

```

15 def _estado_caso(sesion, caso, ejecucion, tolerancia, garantiza_optimo) → str:
16     if ejecucion.estado ≠ 'encontrada':
17         return 'sin_ruta'
18     if ejecucion.ruta[0] ≠ caso.inicio or ejecucion.ruta[-1] ≠ caso.destino:
19         return 'extremos_incorrectos'
20     try:
21         costo_ruta = sesion.costo_ruta(ejecucion.ruta)
22     except ValueError:
23         return 'ruta_invalida'
24     if abs(costo_ruta - ejecucion.costo) > tolerancia:
25         return 'costo_ruta_incorrecto'
26     diferencia = ejecucion.costo - caso.optimo
27     if abs(diferencia) ≤ tolerancia:
28         return 'ok'
29     if diferencia > 0 and not garantiza_optimo:
30         return 'suboptima'      # esperado en BFS y voraz: ruta legal pero más cara
31     return 'costo_referencia_diferente'

```

- 1 No basta con comparar el costo que informa el algoritmo: primero se comprueba que la ruta empiece y termine donde debe.
- 2 `sesion.costo_ruta` vuelve a sumar los pesos paso por paso. Si algún paso no es una arista legal (atraviesa una pared o salta celdas), la ruta es inválida.
- 3 Recién entonces se compara con el óptimo, con una tolerancia de 0.00001 por el redondeo de los decimales publicados.
- 4 Para BFS y voraz, una ruta legal pero más cara se marca `suboptima`: es lo esperado, no un error.

MISIONES CORRECTAS

4 200 / 4 200

ERROR MÁXIMO

 1.1×10^{-7}

MAPAS

3PRUEBAS
AUTOMÁTICAS**38**

MAPA	MISIONES	CORRECTAS	ERROR MÁXIMO	EXPANDIDAS (MEDIANA)
den011d	750	750	3.1×10^{-8}	2 246.5
brc201d	2 090	2 090	1.1×10^{-7}	9 109.5
brc100d	1 360	1 360	9.0×10^{-8}	7 796

La evidencia queda en `engine/results/astar/ : casos.csv` con una fila por misión y `resumen.json` con totales, entorno y huellas SHA-256 del código y de los datos. Se regenera con `npc-nav verificar --`

algoritmo astar (unos 10 minutos).

V

Referencia

Lo que se consulta al trabajar: comandos, pruebas y el vocabulario del curso tal como aparece en el código.

19 Comandos y puesta en marcha

20 Pruebas automatizadas

21 Glosario

22 Referencias

Comandos y puesta en marcha

Primera vez en una PC

- 1 Abrir la carpeta raíz del repositorio en PyCharm.
- 2 *Settings* → *Python* → *Interpreter* → *Add Interpreter* → *Virtualenv*, ubicación `...\engine\.venv`.
- 3 En la terminal de PyCharm: `cd engine` y luego `pip install -e .` (con figuras: `pip install -e "[figures]"`).
- 4 Clic derecho en `engine/src` → *Mark Directory as* → *Sources Root*; en `engine/tests` → *Test Sources Root*.
- 5 *Settings* → *Tools* → *Terminal* → marcar **Activate virtualenv**. Si una terminal no muestra `(.venv)`: `.\.venv\Scripts\Activate.ps1`.

El comando `npc-nav`

SUBCOMANDO	QUÉ HACE	EJEMPLO
<code>algoritmos</code>	Lista el catálogo	<code>npc-nav algoritmos</code>
<code>analizar</code>	Estadísticas del grafo	<code>npc-nav analizar --mapa brc201d</code>
<code>buscar</code>	Un algoritmo, una misión; JSON con traza	<code>npc-nav buscar --mapa den011d --caso 375 --traza --output r.json</code>
<code>comparar</code>	Varios algoritmos, misma misión	<code>npc-nav comparar --mapa den011d --caso 375</code>
<code>validar</code>	Integridad del dataset	<code>npc-nav validar</code>
<code>verificar</code>	Contra las 4 200 misiones	<code>npc-nav verificar --algoritmo astar</code>

Desde Python: un obstáculo dinámico

```

from npc_nav.application import ejecutar_busqueda
from npc_nav.domain import SesionNavegacion
from npc_nav.infrastructure import RepositorioDataset

repo = RepositorioDataset()
caso = repo.escenario('den011d', 375)
sesion = SesionNavegacion(repo.mapa('den011d'))

primera = ejecutar_busqueda(sesion, 'astar', caso.inicio, caso.destino)

# El NPC va por el paso 39 y aparece un bloque en el paso 40: replanifica desde donde está
sesion.actualizar_obstaculos(bloquear=[primera.ruta[40]])
nueva = ejecutar_busqueda(sesion, 'astar', primera.ruta[39], caso.destino)

```

Pruebas automatizadas

38 pruebas comprueban el motor en menos de 2 segundos. PyCharm muestra más (285) porque cuenta cada caso interno, como cada uno de los mapas aleatorios.

ARCHIVO	QUÉ COMPRUEBA
<code>unit/test_sesion.py</code>	8 vecinos con costos 1 y $\sqrt{2}$, no se cortan esquinas, costo $\neq$ pasos, obstáculos atómicos, versión, sesiones aisladas, heurística consistente.
<code>unit/test_algoritmos.py</code>	El contrato común para los 4 algoritmos; Dijkstra y A* contra un oráculo en 40 mapas aleatorios; BFS y voraz más caros en mapas contruidos para demostrarlo.
<code>unit/test_busqueda.py</code>	Métricas y traza coinciden, determinismo, versión del mapa, errores ante datos inválidos.
<code>unit/test_ufds.py</code>	Uniones, representantes, conteo de conjuntos y regiones.
<code>unit/test_movingai.py</code>	Lectura correcta, rechazo de archivos dañados, conversión (x, y) $\rightarrow$ (fila, columna).
<code>integration/test_dataset_real.py</code>	Los 3 mapas reproducen las cifras del informe TB1.
<code>integration/test_validacion_verificacion.py</code>	Detección de datos incoherentes y óptimos falsos; el CLI escribe sus archivos.

¿QUÉ ES EL ORÁCULO?

Para saber si A* es óptimo en un mapa aleatorio hace falta la respuesta correcta. El oráculo es un Dijkstra de 12 líneas escrito dentro de la prueba, independiente del código del proyecto. Si A* coincide con él en 40 mapas distintos, es muy improbable que ambos se equivoquen igual.

Glosario

TÉRMINO	SIGNIFICADO	DÓNDE APARECE
Vértice	Celda caminable	<code>Mapa.posiciones_transitables()</code>
Arista	Movimiento legal entre vecinas, peso 1 o $\sqrt{2}$	<code>movimientos_desde()</code>
Grafo implícito	No se guarda completo; se consulta vecino por vecino	<code>SesionNavegacion.vecinos()</code>
Frontera	Celdas descubiertas pendientes de examinar	Aviso <code>frontera</code> · amarillo
Expandir	Sacar una celda de la frontera y examinar sus vecinos	Aviso <code>expandido</code> · azul
$g \cdot h \cdot f$	Costo recorrido · estimación restante · suma	Todos los algoritmos
Relajación	Mejorar el costo de una celda al hallar un camino más barato	Paso 7 de Dijkstra y A*
Heurística admisible	Nunca sobreestima; garantiza el óptimo de A*	<code>distancia_octil</code>
Heurística consistente	$h(u) \leq w(u, v) + h(v)$; no hay que reabrir celdas	<code>distancia_octil</code>
Montículo mínimo	Entrega el menor elemento en $O(\log n)$	<code>heapq</code>
Cola FIFO	El primero en entrar es el primero en salir	<code>deque</code> en BFS
Algoritmo voraz	Elige lo localmente mejor sin reconsiderar	<code>voraz.py</code>
Componente conexa	Grupo de celdas alcanzables entre sí	<code>ufds.py</code>
Escenario	Misión del benchmark: inicio, destino, óptimo	<code>.scen</code> · Escenario
Traza	Lista ordenada de avisos de una búsqueda	<code>Ejecucion.traza</code>
Observador	Objeto al que el algoritmo le avisa sus pasos	<code>contrato.py</code>
Sesión	Mapa + obstáculos dinámicos de una partida	<code>SesionNavegacion</code>

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press. Sección 22.2.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.

Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
<https://doi.org/10.1109/TSSC.1968.300136>

Koenig, S. y Likhachev, M. (2002). D* Lite. En *Proceedings of the Eighteenth National Conference on Artificial Intelligence* (pp. 476–483). AAAI.

Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson. Sección 3.5.1.

Sturtevant, N. R. (2012). Benchmarks for grid-based pathfinding. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2), 144–148. <https://doi.org/10.1109/TCIAIG.2012.2197681>

Tarjan, R. E. (1975). Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2), 215–225.

Moving AI Lab. *2D pathfinding benchmarks*. Universidad de Denver.
<https://www.movingai.com/benchmarks/>